

OOP

UNIT 7 SUPPLEMENT -- STATIC AND DYNAMIC BINDING AND THE VIRTUAL TABLE

A pointer only ever promises the type it was declared with; the object behind it may promise more.

INTRODUCTION

Unit 7 showed that a non-virtual function call through a base pointer runs the base class's version, while a virtual function call through the same pointer runs whichever version the actual object overrides -- and named the mechanism dynamic binding.

This supplement gives that behavior its full vocabulary and explains how the compiler actually implements it: through a hidden per-class table of function pointers called the virtual table, or vtable.

STATIC BINDING: RESOLVED AT COMPILE TIME

Static binding -- also called *early* binding -- means the **compiler** decides which function a call invokes while compiling the program, using only the declared type of the pointer or reference through which the call is made.

```
class Song {
public:
    void play() const {
        cout << "Playing a song." << endl;
    };
};
class LiveRecording : public Song {
public:
    void play() const {
        cout << "Playing a live recording." << endl;
    };
};

int main() {
    LiveRecording concert;
    Song* base = &concert;
    base->play();
    concert.play();
    return 0; }
```

The program prints:

```
Playing a song.  
Playing a live recording.
```

base is declared as `Song*`, so `base->play()` is compiled as a direct call to `Song::play`, regardless of what base happens to point to when the program runs. It does not matter that base actually points to a `LiveRecording` -- `LiveRecording::play` here is an entirely separate, unrelated function that happens to share a name, not an override.

Calling `play` directly on `concert`, whose *declared* type is `LiveRecording`, calls `LiveRecording::play` as expected.

Every function call in this example is a *plain*, direct call, *fixed* at *compile* time -- which is what makes static binding both fast and, when a hierarchy is meant to be polymorphic, easy to get wrong by accident.

DYNAMIC BINDING: RESOLVED AT RUN TIME

Dynamic binding -- also called *late* binding -- defers that decision to *run time*, and bases it on the dynamic type of the object the pointer or reference actually refers to, not on the pointer's own declared type. Marking a function *virtual* is what asks the compiler to generate this run-time lookup instead of a fixed call.

```
class Song {
public:
    virtual void play() const    {
        cout << "Playing a song." << endl;
    }

    virtual ~Song() {}
};

class LiveRecording : public Song {
public:
    void play() const override {
        cout << "Playing a live recording." << endl;
    }
};
```

```
int main() {  
    LiveRecording concert;  
    Song* base = &concert;  
  
    base->play();  
    return 0;  
}
```

The program prints:

```
Playing a live recording.
```

The only change from the previous example is the *virtual* keyword on `Song::play` and *override* on `LiveRecording::play`. `base` is still declared as `Song*`, but the call now resolves to `LiveRecording::play`, because *dynamic* binding looks at what `base` actually points to -- a `LiveRecording` -- rather than at how `base` is declared. This is the behavior Unit 7 relied on throughout; the next section explains how the compiler makes it happen.

HOW DYNAMIC BINDING IS IMPLEMENTED: THE VIRTUAL TABLE

A direct call, as in static binding, compiles down to a single machine instruction: jump to this fixed address. Dynamic binding cannot compile down to a fixed address, because which function to run is not known until run time -- it depends on the object, and the same line of code can end up calling different functions depending on what it is pointed at. The compiler solves this with one extra layer of indirection.

For every class that declares or inherits at least one *virtual* function, the compiler generates a *virtual* table: a hidden array of function pointers, one entry per virtual function, each entry holding the address of whichever override actually applies to that class. Every object of such a class carries one hidden pointer, usually called the *vp*tr, that points to its own class's virtual table. The *vp*tr is set automatically during construction and never has to be written by hand.

A virtual call such as `base->play()` compiles into three steps instead of one: follow base to the object, follow the object's *vp*tr to its class's vtable, and call whichever function address sits in play's slot of that table. Because a `LiveRecording`'s *vp*tr points to `LiveRecording`'s vtable, and that table's play slot holds the address of `LiveRecording::play`, the call reaches `LiveRecording::play` even though base is declared as `Song*`. Nothing about base's declared type enters into the lookup at all -- *only the vp*tr the object itself carries.

The *vptr* is not free. It is a real hidden member that every polymorphic object carries, and it makes such objects slightly larger than an otherwise identical class with no virtual functions.

```
class PlainSong {
private:
    string title;
public:
    PlainSong(string title) : title(title) {}
    void play() const      { cout << "Playing " << title << endl; };
class PolymorphicSong {
private:
    string title;
public:
    PolymorphicSong(string title) : title(title) {}
    virtual void play() const      { cout << "Playing " << title << endl; }
    virtual ~PolymorphicSong() {}
};
int main() {
    cout << "sizeof(PlainSong): " << sizeof(PlainSong) << endl;
    cout << "sizeof(PolymorphicSong): " << sizeof(PolymorphicSong) << endl;
    return 0;
}
```

On the 64-bit system used to test this example, the program prints:

```
sizeof(PlainSong): 32  
sizeof(PolymorphicSong): 40
```

PlainSong and PolymorphicSong store the exact same data -- a single string title -- and differ only in that PolymorphicSong declares a virtual function.

The eight-byte difference is the *vptr*: the size of one pointer on a 64-bit system, added once per object, invisible in the source code, and present in every object of a class with at least one virtual function.

This is the concrete, physical cost of making dynamic binding possible -- one extra pointer per object, and one extra indirection per virtual call.

COMMON MISTAKE: EXPECTING DYNAMIC BINDING INSIDE A CONSTRUCTOR

The *vp*tr is set during construction, which means it does not necessarily hold its final value throughout the entire construction process. This produces a well-known surprise: a virtual call made from inside a base class's constructor does not dispatch to a derived class's override, even when the object under construction is about to become that derived type.

```
class AudioTrack {
public:
    AudioTrack() {
        announce(); //this announce belongs to AudioTrack (base)
    }
    virtual void announce() const {
        cout << "Constructing an AudioTrack." << endl;
    }
    virtual ~AudioTrack() {}
};

class Song : public AudioTrack {
public:
    void announce() const override {
        cout << "Constructing a Song." << endl;
    }
};
```

```
int main()
{
    Song s;

    return 0;
}
```

Song inherits announce from AudioTrack and overrides it. A programmer expecting dynamic binding might expect constructing a Song to print "Constructing a Song." Instead, the program prints:

```
Constructing an AudioTrack.
```

Recall from Unit 7 that

constructing a Song first runs AudioTrack's constructor,
and only afterward runs Song's own constructor body.

While AudioTrack's constructor is running, the object's vptr has been set to point at AudioTrack's vtable -- Song's constructor has not run yet, so the object is not yet, in any observable sense, a Song. A virtual call made at that moment can only reach the overrides that exist so far, which are AudioTrack's own. The vptr is updated to point at Song's vtable only once Song's constructor begins, which is always after AudioTrack's constructor has already finished running. The same reasoning applies in reverse during destruction: a virtual call made from a base class's destructor reaches the base class's own override, because the derived class's destructor has already run and the vptr has already been moved back. The safe rule is simple: never rely on a virtual call inside a constructor or destructor to reach an override defined further down the hierarchy than the class currently under construction or destruction.

SUMMARY

Static binding resolves a function call at *compile* time, using only the declared type of the pointer or reference making the call, and compiles down to a single direct call.

Dynamic binding defers that resolution to run time and bases it on the *actual*, dynamic type of the object involved.

The compiler implements dynamic binding through the *virtual table*: a per-class array of function pointers, reached through a hidden *vp*tr that every polymorphic object carries and that construction sets automatically. That *vp*tr adds a small amount to an object's size.

VPTR is not set until the object's own constructor executed -- which is why a virtual call made from inside a base class's constructor or destructor still resolves to that base class's own version, not to any override defined further down the hierarchy (don't call virtual functions from constructor\destructor).